

MidEng 7.3 Message Oriented Middleware (Grundlagen plus Vertiefung)

DEZSYS_GK73_WAREHOUSE_MOM

Verfasser: Ramis Ekici

Datum: 27.11.2025

Theorie

Apache Kafka

Apache Kafka ist eine **verteilte Streaming- und Messaging-Plattform**, die verwendet wird, um **Daten in Echtzeit** zwischen Systemen zu übertragen.

Man kann sich Kafka wie eine **riesige, extrem schnelle Poststelle** vorstellen:

- Produzenten schicken Nachrichten hinein,
- Kafka speichert sie,
- Konsumenten holen sie ab.

Kafka ist besonders gut geeignet, wenn viele Daten schnell und zuverlässig transportiert werden müssen (z. B. Sensor-Daten, Log-Daten, Bestellungen, Lagerupdates, etc.).

Wofür?

Daten zwischen Systemen schicken

Z. B. Microservices, Server, Anwendungen, IoT-Geräte, Sensoren.

Echtzeit-Verarbeitung

Kafka kann Millionen Nachrichten **pro Sekunde** transportieren.

Daten speichern (temporär)

Kafka speichert Nachrichten **für eine bestimmte Zeit** (z. B. 7 Tage).

Systeme entkoppeln

Producer und Consumer müssen sich **nicht kennen**.

Event-basierte Architektur

Systeme reagieren auf Ereignisse („Events“), die über Kafka kommen.

Kafka Komponenten

Producer (Erzeuger)

Schickt Nachrichten an Kafka.

MessageProducer (Warehouse sendet Daten).

Consumer (Verbraucher)

Liest Nachrichten aus Kafka.

MessageConsumer (nimmt Warehouse-Daten entgegen).

Topics

Themen-Kanäle – wie „Ordner“, wo Nachrichten hineingelegt werden.

Beispiele:

- warehouse-wien
- warehouse-reserve

Producer schreibt → Topic → Consumer liest

Partitions

Jedes Topic kann in mehrere Teile aufgeteilt werden.

Vorteile:

- Parallelität
- Höhere Geschwindigkeit
- Skalierbarkeit

Broker

Ein Kafka-Server, der Nachrichten speichert.

Kafka-Cluster = viele Broker zusammen.

Consumer Group

Mehrere Consumer arbeiten zusammen.

Kafka verteilt die Nachrichten dann automatisch auf die Gruppe.

Zookeeper (älter, wird weniger verwendet)

Diente früher zur Verwaltung des Kafka-Clusters.

Heute gibt es Kafka **ohne Zookeeper**: KIP-500.

Praxis

Kafka Einstellen

Zuerst muss **Apache Kafka** vom Docker installiert werden. Also container pullen und image erstellen. Danach kann man mit folgenden Befehlen arbeiten.

Um auf **Kafka Terminal** zu arbeiten:

```
> docker exec -it containerName bash
> cd /opt/kafka
```

Topic erstellen / zeigen:

```
kafka-topics.sh --create --topic topicName --bootstrap-server localhost:9092

kafka-topics.sh --list --bootstrap-server localhost:9092

kafka-topics.sh --describe --topic topicName --bootstrap-server localhost:9092
```

Nachrichten **Testen**:

```
# Producer
kafka-console-producer.sh --topic warehouse-wien --bootstrap-server localhost:9092
# Consumer
kafka-console-consumer.sh --topic warehouse-wien --from-beginning --bootstrap-server localhost:9092
```

Gruppen:

```
kafka-console-consumer.sh --topic warehouse-wien --from-beginning --bootstrap-server localhost:9092
```

```
kafka-consumer-groups.sh --bootstrap-server localhost:9092 --group warehouse-group --describe
```

Kafka-Container Logs:

```
docker logs kafka
docker logs zookeeper
```

Model Klassen

Zuerst brauchen wir natürlich die Daten selber also die Modelklassen. In meinem Fall sind diese: **ProductData, WarehouseData**

Producer

In diesem Fall brauchen wir eine Klasse oder ein Objekt von Java Spring Boot Framework. Und zwar den **KafkaTemplate**. Das KafkaTemplate kapselt einen Producer und stellt praktische Methoden zum Senden von Nachrichten an Kafka-Topics bereit.

Mehr dazu kann man hier finden:

[KafkaTemplate \(Spring for Apache Kafka 4.0.0 API\)](#)

Logischerweise muss dann eine Post Methode zum Schicken der Daten benötigt.

```
// Dabei muss diese Objekt mit @Autowired injktiert werden
@Autowired
private KafkaTemplate<String, WarehouseData> kafkaTemplate;

@PostMapping("/send")
public String sendMessage(@RequestBody WarehouseData warehouseData) {
    // sende die Daten in angegebenen Topic
    kafkaTemplate.send("warehouse-wien", warehouseData);
    return "SUCCESS";
}
```

Service

Vor der Implementierung des Consumers, beginnen wir mit der Service Klasse, um alle Warehouses permanent speichern zu können (persistieren fehlt also Datenbank zb.).

```
// Thread-safe Map für alle Lagerstandorte
private final Map<String, WarehouseData> allWarehouses = new ConcurrentHashMap<>();

public void addOrUpdateWarehouse(WarehouseData warehouseData) {
    allWarehouses.put(warehouseData.getWarehouseID(), warehouseData);
}

public Map<String, WarehouseData> getAllWarehouses() {
    return allWarehouses;
}
```

Consumer

Nun kann man mit Consumer beginnen. Für diese Aufgabe nutzen ein altes Wissen aus dritten Klasse das Arbeiten mit Textdateien ([FileWriter \(Java Platform SE 8 \)](#)). Ansonsten wird in dieser Klasse ebenso ein [KafkaTemplate](#) und [KafkaListener \(Spring for Apache Kafka 4.0.0 API\)](#) verwendet.

Er hört automatisch auf Nachrichten, die in einem bestimmten **Kafka-Topic** ankommen, und führt eine Methode aus, sobald eine neue Nachricht eintrifft.

```
messageService.addOrUpdateWarehouse(warehouseData);
String feedbackTopic = "warehouse-reserve"; // Topic für Rückmeldung
String ausgabe = "SUCCESS: " + warehouseData.getWarehouseID() + ", " +
warehouseData.getWarehouseName();
kafkaTemplate.send(feedbackTopic, warehouseData.getWarehouseID(), "SUCCESS: " +
warehouseData.getWarehouseID() + ", " + warehouseData.getWarehouseName());
```

- `messageService.addOrUpdateWarehouse(warehouseData);`
Speichert den Lagerstand (WarehouseData) in einer internen Map oder aktualisiert ihn dort.
- `String feedbackTopic = "warehouse-reserve";`
Definiert das Kafka-Topic, an das eine Rückmeldung geschickt wird.
- `String ausgabe = "...";`
Baut einen kurzen Erfolgstext zusammen.
- `kafkaTemplate.send(...)`
Schickt eine **Erfolgsmeldung zurück an Kafka**, damit andere Services wissen:
„Dieses Warehouse wurde erfolgreich verarbeitet.“

Diese Methode ist vorteilhafter als dynamisch mit mehreren Topics zu arbeiten, weil wir in dem Fall so viele Topics wie wir wollen erstellen können.

Controller

Eine Klasse wo die Endpunkte legen, um die gespeicherten Daten im Browser aufrufen zu können. Wie in der Rest Aufgabe.

FeedbackConsumer

Diese Klasse hört auf das Kafka-Topic `warehouse-reserve` und druckt jede empfangene Nachricht auf der Konsole aus.

```
@KafkaListener(topics = "warehouse-reserve")
public void receiveFeedback(String feedback) {
    System.out.println("Received feedback: " + feedback);
}
```

Quelle

[1] \tab \uc0\u8222{}FileWriter (Java Platform SE 8)\uc0\u8220{}. Zugegriffen: 27. November 2025. [Online]. Verf\uc0\u252{}gbar unter: <https://docs.oracle.com/javase/8/docs/api/java/io/FileWriter.html>

\

[2] \tab \uc0\u8222{}KafkaTemplate (Spring for Apache Kafka 4.0.0 API)\uc0\u8220{}. Zugegriffen: 27. November 2025. [Online]. Verf\uc0\u252{}gbar unter: <https://docs.spring.io/spring-kafka/api/org/springframework/kafka/core/KafkaTemplate.html>

\

[3] \tab \uc0\u8222{}KafkaListener (Spring for Apache Kafka 4.0.0 API)\uc0\u8220{}. Zugegriffen: 27. November 2025. [Online]. Verf\uc0\u252{}gbar unter: [https://docs.spring.io/spring-kafka/api/org/springframework/kafka/annotation/KafkaListener.html#topics\(\)](https://docs.spring.io/spring-kafka/api/org/springframework/kafka/annotation/KafkaListener.html#topics())

\

[4] \tab \uc0\u8222{}Codemia | KafkaListener concurrency multiple topics\uc0\u8220{}. Zugegriffen: 27. November 2025. [Online]. Verf\uc0\u252{}gbar unter: https://codemia.io/knowledge-hub/path/kafkalistener_concurrency_multiple_topics

\

[5] \tab \uc0\u8222{}Apache Kafka and Java - Getting Started Tutorial\uc0\u8220{}, Confluent. Zugegriffen: 27. November 2025. [Online]. Verf\uc0\u252{}gbar unter: <https://developer.confluent.io/get-started/java/>

\

[6] \tab \uc0\u8222{}Apache Kafka\uc0\u8220{}, Apache Kafka. Zugegriffen: 27. November 2025.
[Online]. Verf\uc0\u252{}gbar unter: <https://kafka.apache.org/quickstart>

\
}